Intelligent Retail Decision Assistant - Analysis (Elias)

Summary

An end-to-end retail prototype: classical ML on 1,500 interaction records (purchase and return prediction, customer price-sensitivity segments), a FAISS-backed RAG layer over 1,497 product listings and 5 policies, and an inspectable agent that routes requests to five tools. Best purchase model: Random Forest (test ROC-AUC 0.67). Most price-sensitive segment: Deal Seekers (price-sensitive). Assistant: 100% of 16 curated queries passed, 100% grounded and validly cited (backend: offline).

Design

data_prep -> supervised / clustering -> documents -> embeddings + vector_store (FAISS) -> rag (product) / policy_qa -> agent (guardrails, rule router, LLM fallback, trace) -> eval / app CLI. LLM: gpt-4o-mini, temperature 0, JSON output; embeddings text-embedding-3-small via the Vocareum OpenAI-compatible endpoint. Every LLM output is validated against retrieved context; a deterministic fallback keeps the assistant working if the API fails.

Results

Component	Metric	Result
Purchase model	Random Forest test ROC-AUC / F1 / PR-AUC	0.673 / 0.649 / 0.685
Return model	Random Forest test ROC-AUC / PR-AUC (base 0.35)	0.611 / 0.418
Segmentation	K-Means k=4 silhouette / ARI vs Ward	0.283 / 0.49
Product retrieval	precision@5 hybrid vs semantic	0.99 vs 0.74
Product retrieval	MRR hybrid vs semantic	1.00 vs 0.80
Assistant	pass rate / route accuracy	100% / 100%
Assistant	citation validity / grounded rate	100% / 100%
Tests	pytest (offline, fake LLM)	46 passed in 20.54s

Key findings (including what failed and how it was fixed)

- Pandas silently converted the literal 'NA' in size (270 rows) to NaN; fixed by reading with keep_default_na=False and treating it as an explicit missing value imputed as 'Unknown'.

- Leakage found: review_sentiment_label is a direct mapping of rating_given, and rating 5 always means purchased; rating/sentiment, returned_flag and checkout fields were excluded from the purchase model.

- First clustering attempt on spending-history features gave silhouette ~0.12 because customer averages of order value / order count barely differ between customers; switching to behavioural price-sensitivity features raised it to 0.28.

- Engineered features (price-to-AOV, deep-discount flag, premium member, savings) helped the returns model (CV AUC 0.618 -> 0.635) but not the purchase model, so they are used only for returns.

- 'gift card' initially matched wallets (via 'card holder'); 'cancellation policy' matched the warranty policy through the generic word 'policy'. Fixed with a confidence rule, stop-words and thresholds - both are now refused instead of answered.

- Filter relaxation could return caps for a watch request; now only brand/colour are relaxed and a no-match answer states the cheapest available price.

- A fake model returning an invented product id, a wrong price and a fake policy citation was used in tests: the validators dropped the id, restored the catalogue price and refused the uncited answer.

- pytest initially overwrote results/; tests now write to a temporary copy.

Limitations

- The dataset is synthetic: the same product_id maps to different items and customer attributes vary between rows, which caps predictive accuracy (purchase ROC-AUC ~0.67).

- Policies are five short synthetic snippets; real policy documents would need longer chunking.

- The router is rule-based with an LLM fallback only when no rule fires; unusual phrasing may need new patterns.

- Offline mode (TF-IDF + template answers) is a deterministic stand-in for tests; final answers should be produced with the API backend.

- No image understanding (optional multimodal extension not implemented).

Acceptance criteria

Requirement	Evidence	Status
Main notebook	Capstone_Graded Project_Elias.ipynb (all phases, outputs saved)	PASS
Dataset + reproduction of added columns	data/retail_interactions.csv, data_prep.ENGINEERED, results/data_dictionary.csv	PASS
Data dictionary	results/data_dictionary.csv (46 columns, source + how reproduced)	PASS

Supervised ML results	results/ml_model_comparison.csv - purchase Random Forest test ROC-AUC 0.673	PASS
Unsupervised ML results	results/segment_profiles.csv - k=4, silhouette 0.283	PASS
Product retrieval / recommendation	rag.py + FAISS; hybrid precision@5 0.99	PASS
Policy QA with citations	policy_qa.py; Q02, Q06-Q10 in results/eval_results.csv	PASS
Agent / controller demo with trace	agent.py; results/demo_runs.json, fig11_demo_*.png	PASS
At least five example queries	16 curated queries, pass rate 100%	PASS
Regression-like checks + tests	results/regression_checks.csv; pytest: 46 passed in 20.54s	PASS
Report / README	README.md, Capstone_Report_Elias.docx, Capstone_Analysis_Elias.pdf	PASS